

Completing the Rebase of Library Fundamentals, Version 3, Working Draft

Doc. no.	P1210R0
Date:	2018-09-30
Project:	Programming Language C++
Reply to:	Alisdair Meredith < ameredith1@bloomberg.net >
Audience:	LEWG, Library

Table of Contents

- 0. [Revision History](#)
 - [Revision 0](#)
- 1. [Introduction](#)
- 2. [Problems to be addressed](#)
- 3. [Proposed Resolution](#)
- 4. [Alternatives Considered](#)
- 5. [Formal Wording](#)
- 6. [Acknowledgements](#)
- 7. [References](#)

1. Introduction

There were a few small issues applying [P0996R1](#) to the Library Fundamentals TS working draft, to rebase it onto C++17. This paper resolves any outstanding issues cleaning up from the rebase.

2. Introduction

When [P0996R1](#) was adopted to update the Library Fundamentals TS for C++17, there was a clear intent to remove all the variable templates for type traits that has already landed in C++17. Indeed, the feature macro for the variable template traits is removed from the set of feature macros associated with this TS. However, the actual wording change to remove the variable templates was missing.

In addition, it was not noticed that several new variable templates are added to support new type traits proposed by the TS. C++17 added the notion of inline variables, and variable template for type traits are consistently specified using the `inline` keyword in C++17.

3. Proposed Resolution

There are two parts to the proposed resolution. First, remove all the variable templates for traits that have landed in the main standard, ISO 14882:2017. Secondly, apply the `inline` keyword to variable templates defined in the fundamentals TS, consistent with the usage for variable template traits in ISO 14882:2017.

4. Alternatives Considered

We could just leave the variable templates as part of the experimental namespace, as they would remain just as valid. However, apart from adding noise for a feature that has already landed in the main standard, the set of variable templates is incomplete, as new features continue to land in the main standard and would not be backported into the

experimental namespace. The original intent was to remove features from the TS once they were no longer experimental, and we see no reason to be selectively different in this case.

5. Formal Wording

Apply the following changes to the (proposed) Library Fundamentals V3 TS working draft. These changes are relative to [N4758](#).

3 General utilities library

[utilities]

3.3 Metaprogramming and type traits

[meta]

3.3.1 Header <experimental/type_traits> synopsis

[meta.type.synop]

```
#include <type_traits>

namespace std::experimental {
inline namespace fundamentals_v3 {

    // See C++17 §23.15.4.1, primary type categories
    template <class T> constexpr bool is_void_v
        = is_void<T>::value;
    template <class T> constexpr bool is_null_pointer_v
        = is_null_pointer<T>::value;
    template <class T> constexpr bool is_integral_v
        = is_integral<T>::value;
    template <class T> constexpr bool is_floating_point_v
        = is_floating_point<T>::value;
    template <class T> constexpr bool is_array_v
        = is_array<T>::value;
    template <class T> constexpr bool is_pointer_v
        = is_pointer<T>::value;
    template <class T> constexpr bool is_lvalue_reference_v
        = is_lvalue_reference<T>::value;
    template <class T> constexpr bool is_rvalue_reference_v
        = is_rvalue_reference<T>::value;
    template <class T> constexpr bool is_member_object_pointer_v
        = is_member_object_pointer<T>::value;
    template <class T> constexpr bool is_member_function_pointer_v
        = is_member_function_pointer<T>::value;
    template <class T> constexpr bool is_enum_v
        = is_enum<T>::value;
    template <class T> constexpr bool is_union_v
        = is_union<T>::value;
    template <class T> constexpr bool is_class_v
        = is_class<T>::value;
    template <class T> constexpr bool is_function_v
        = is_function<T>::value;

    // See C++17 §23.15.4.2, composite type categories
    template <class T> constexpr bool is_reference_v
        = is_reference<T>::value;
    template <class T> constexpr bool is_arithmetic_v
        = is_arithmetic<T>::value;
    template <class T> constexpr bool is_fundamental_v
        = is_fundamental<T>::value;
    template <class T> constexpr bool is_object_v
        = is_object<T>::value;
    template <class T> constexpr bool is_scalar_v
        = is_scalar<T>::value;
    template <class T> constexpr bool is_compound_v
        = is_compound<T>::value;
    template <class T> constexpr bool is_member_pointer_v
        = is_member_pointer<T>::value;
```

```

// See C++17 §23.15.4.3, type properties
template <class T> constexpr bool is_const_v
    = is_const<T>::value;
template <class T> constexpr bool is_volatile_v
    = is_volatile<T>::value;
template <class T> constexpr bool is_trivial_v
    = is_trivial<T>::value;
template <class T> constexpr bool is_trivially_copyable_v
    = is_trivially_copyable<T>::value;
template <class T> constexpr bool is_standard_layout_v
    = is_standard_layout<T>::value;
template <class T> constexpr bool is_pod_v
    = is_pod<T>::value;
template <class T> constexpr bool is_literal_type_v
    = is_literal_type<T>::value;
template <class T> constexpr bool is_empty_v
    = is_empty<T>::value;
template <class T> constexpr bool is_polymorphic_v
    = is_polymorphic<T>::value;
template <class T> constexpr bool is_abstract_v
    = is_abstract<T>::value;
template <class T> constexpr bool is_final_v
    = is_final<T>::value;
template <class T> constexpr bool is_signed_v
    = is_signed<T>::value;
template <class T> constexpr bool is_unsigned_v
    = is_unsigned<T>::value;
template <class T, class... Args> constexpr bool is_constructible_v
    = is_constructible<T, Args...>::value;
template <class T> constexpr bool is_default_constructible_v
    = is_default_constructible<T>::value;
template <class T> constexpr bool is_copy_constructible_v
    = is_copy_constructible<T>::value;
template <class T> constexpr bool is_move_constructible_v
    = is_move_constructible<T>::value;
template <class T, class U> constexpr bool is_assignable_v
    = is_assignable<T, U>::value;
template <class T> constexpr bool is_copy_assignable_v
    = is_copy_assignable<T>::value;
template <class T> constexpr bool is_move_assignable_v
    = is_move_assignable<T>::value;
template <class T> constexpr bool is_destructible_v
    = is_destructible<T>::value;
template <class T, class... Args> constexpr bool is_trivially_constructible_v
    = is_trivially_constructible<T, Args...>::value;
template <class T> constexpr bool is_trivially_default_constructible_v
    = is_trivially_default_constructible<T>::value;
template <class T> constexpr bool is_trivially_copy_constructible_v
    = is_trivially_copy_constructible<T>::value;
template <class T> constexpr bool is_trivially_move_constructible_v
    = is_trivially_move_constructible<T>::value;
template <class T, class U> constexpr bool is_trivially_assignable_v
    = is_trivially_assignable<T, U>::value;
template <class T> constexpr bool is_trivially_copy_assignable_v
    = is_trivially_copy_assignable<T>::value;
template <class T> constexpr bool is_trivially_move_assignable_v
    = is_trivially_move_assignable<T>::value;
template <class T> constexpr bool is_trivially_destructible_v

```

```

    = is_trivially_destructible<T>::value;
template <class T, class... Args> constexpr bool is_nothrow_constructible_v
    = is_nothrow_constructible<T, Args...>::value;
template <class T> constexpr bool is_nothrow_default_constructible_v
    = is_nothrow_default_constructible<T>::value;
template <class T> constexpr bool is_nothrow_copy_constructible_v
    = is_nothrow_copy_constructible<T>::value;
template <class T> constexpr bool is_nothrow_move_constructible_v
    = is_nothrow_move_constructible<T>::value;
template <class T, class U> constexpr bool is_nothrow_assignable_v
    = is_nothrow_assignable<T, U>::value;
template <class T> constexpr bool is_nothrow_copy_assignable_v
    = is_nothrow_copy_assignable<T>::value;
template <class T> constexpr bool is_nothrow_move_assignable_v
    = is_nothrow_move_assignable<T>::value;
template <class T> constexpr bool is_nothrow_destructible_v
    = is_nothrow_destructible<T>::value;
template <class T> constexpr bool has_virtual_destructor_v
    = has_virtual_destructor<T>::value;

// See C++17 §23.15.5, type property queries
template <class T> constexpr size_t alignment_of_v
    = alignment_of<T>::value;
template <class T> constexpr size_t rank_v
    = rank<T>::value;
template <class T, unsigned I = 0> constexpr size_t extent_v
    = extent<T, I>::value;

// See C++17 §23.15.6, type relations
template <class T, class U> constexpr bool is_same_v
    = is_same<T, U>::value;
template <class Base, class Derived> constexpr bool is_base_of_v
    = is_base_of<Base, Derived>::value;
template <class From, class To> constexpr bool is_convertible_v
    = is_convertible<From, To>::value;

// 3.3.2, Other type transformations
template <class> class invocation_type; // not defined
template <class F, class... ArgTypes> class invocation_type<F(ArgTypes...)>;
template <class> class raw_invocation_type; // not defined
template <class F, class... ArgTypes> class raw_invocation_type<F(ArgTypes...)>;

template <class T>
    using invocation_type_t = typename invocation_type<T>::type;
template <class T>
    using raw_invocation_type_t = typename raw_invocation_type<T>::type;

// 3.3.3, Detection idiom
template <class...> using void_t = void;

struct nonesuch {
    nonesuch() = delete;
    ~nonesuch() = delete;
    nonesuch(nonesuch const&) = delete;

    void operator=(nonesuch const&) = delete;
};

template <template<class...> class Op, class... Args>

```

```

    using is_detected = see below;
template <template<class...> class Op, class... Args>
    inline constexpr bool is_detected_v = is_detected<Op, Args...>::value;
template <template<class...> class Op, class... Args>
    using detected_t = see below;
template <class Default, template<class...> class Op, class... Args>
    using detected_or = see below;
template <class Default, template<class...> class Op, class... Args>
    using detected_or_t = typename detected_or<Default, Op, Args...>::type;
template <class Expected, template<class...> class Op, class... Args>
    using is_detected_exact = is_same<Expected, detected_t<Op, Args...>>;
template <class Expected, template<class...> class Op, class... Args>
    inline constexpr bool is_detected_exact_v
        = is_detected_exact<Expected, Op, Args...>::value;
template <class To, template<class...> class Op, class... Args>
    using is_detected_convertible = is_convertible<detected_t<Op, Args...>, To>;
template <class To, template<class...> class Op, class... Args>
    inline constexpr bool is_detected_convertible_v
        = is_detected_convertible<To, Op, Args...>::value;

} // inline namespace fundamentals_v3
} // namespace std::experimental

```

4 Function objects

[func]

4.1 Header <experimental/functional> synopsis

[header.functional.synop]

```
#include <functional>

namespace std {
    namespace experimental {
        inline namespace fundamentals_v3 {

            // See C++17 §, Function object binders
            template <class T> constexpr bool is_bind_expression_v
                = is_bind_expression<T>::value;
            template <class T> constexpr int is_placeholder_v
                = is_placeholder<T>::value;

            // 4.2, Class template function
            template<class> class function; // undefined
            template<class R, class... ArgTypes> class function<R(ArgTypes...)>;

            template<class R, class... ArgTypes>
            void swap(function<R(ArgTypes...)>&, function<R(ArgTypes...)>&);

            template<class R, class... ArgTypes>
            bool operator==(const function<R(ArgTypes...)>&, nullptr_t) noexcept;
            template<class R, class... ArgTypes>
            bool operator==(nullptr_t, const function<R(ArgTypes...)>&) noexcept;
            template<class R, class... ArgTypes>
            bool operator!=(const function<R(ArgTypes...)>&, nullptr_t) noexcept;
            template<class R, class... ArgTypes>
            bool operator!=(nullptr_t, const function<R(ArgTypes...)>&) noexcept;

        } // namespace fundamentals_v3
    } // namespace experimental

    template<class R, class... ArgTypes, class Alloc>
    struct uses_allocator<experimental::function<R(ArgTypes...)>, Alloc>;

} // namespace std
```

5 Memory

[memory]

5.1 Header <experimental/memory> synopsis

[header.memory.synop]

```
#include <memory>

namespace std {
    namespace experimental {
        inline namespace fundamentals_v3 {

            // See C++17 §23.10.7, uses_allocator
            template <class T, class Alloc> constexpr bool uses_allocator_v
            = uses_allocator<T, Alloc>::value;

            // 5.2, Non-owning pointers
            template <class W> class observer_ptr;

            // 5.2.6, observer_ptr specialized algorithms
            template <class W>
            void swap(observer_ptr<W>&, observer_ptr<W>&) noexcept;
            template <class W>
            observer_ptr<W> make_observer(W*) noexcept;
            // (in)equality operators
            template <class W1, class W2>
            bool operator==(observer_ptr<W1>, observer_ptr<W2>);

            template <class W1, class W2>
            bool operator!=(observer_ptr<W1>, observer_ptr<W2>);
            template <class W>
            bool operator==(observer_ptr<W>, nullptr_t) noexcept;
            template <class W>
            bool operator!=(observer_ptr<W>, nullptr_t) noexcept;
            template <class W>
            bool operator==(nullptr_t, observer_ptr<W>) noexcept;
            template <class W>
            bool operator!=(nullptr_t, observer_ptr<W>) noexcept;
            // ordering operators
            template <class W1, class W2>
            bool operator<(observer_ptr<W1>, observer_ptr<W2>);
            template <class W1, class W2>
            bool operator>(observer_ptr<W1>, observer_ptr<W2>);
            template <class W1, class W2>
            bool operator<=(observer_ptr<W1>, observer_ptr<W2>);
            template <class W1, class W2>
            bool operator>=(observer_ptr<W1>, observer_ptr<W2>);

        } // inline namespace fundamentals_v3
    } // namespace experimental

    // ,
    template<class T> struct hash<experimental::shared_ptr<T>>;

    // 5.2.7, observer_ptr hash support
    template <class T> struct hash;
    template <class T> struct hash<experimental::observer_ptr<T>>;
```



```
} // namespace std
```

6. Acknowledgements

Thanks to Thomas Köppe for taking up the mantle as project editor for the Library Fundamentals TS, and for highlighting this issue for my attention.

7. References

- [P0996R1](#) Rebase Library Fundamentals TS on C++17
- [N4758](#) Library Fundamentals V3 TS (proposed working draft)